软件仓库管理：从项目快照到 AI 协作

课程：南京大学《生成式软件工程》2026 年第 3 讲；主讲：蒋炎岩。

视频：https://www.bilibili.com/video/BV1kybV6DE47/，时长 1:40:14。

本文依据完整原视频、本地音轨转录、选定画面及官方讲义重组。配图来自本次实际下载的视频，均给出来源时间窗口；音频转录采用真实的 20 秒窗口，时间戳不代表逐字强制对齐。本文是经过主题覆盖审查的学习讲义，不是逐字稿，也不声称逐帧检查全部视频。命令示例凡属编辑补充均明确标注。

学习目标

读完本讲，应能解释项目为什么需要快照，区分工作区、暂存区与历史，判断一次提交的边界，并说明 AI 生成能力提高后为什么仍需要架构设计、验证和版本管理。

开场：AI 增强执行力，为什么仍然要学基础

开场延续前两讲：把 AI 放进真实生活流程，可以把反复执行的步骤沉淀为 skills。但“能够执行”不等于“已经理解人的意图”。同样一句“做一个好用的应用”，不同的人心里可能对应完全不同的界面和使用方式，模型只能根据已有上下文作出选择。

讲师用当时的模型体验、空间推理和 3D 打印案例说明能力进步带来的冲击。这些属于课堂中的个人观察，不能据此推出永久有效的模型排名，更不能把讲师对模型训练路线的猜测改写成厂商事实。

学习操作系统、数据库等基础仍有现实价值：这些知识提供接口、状态、并发和约束的概念，使人能够提出更准确的问题，也能识别方案的不足。基础知识会影响我们向 AI 提什么要求、允许什么取舍，以及怎样判断结果，而不只是影响能否亲手敲出代码。

1. 为什么只有一个项目目录还不够

软件项目通常以目录的形式出现：代码实现行为，文档表达需求与约定，测试描述可以检查的要求，配置和构建脚本决定如何运行。保留当前目录可以保存当下的工作，却不能自动回答“昨天能运行的版本在哪里”。

复制“作业-v1”“作业-v2”是一种朴素的版本管理方法。它能留下旧文件，但版本关系需要人工记忆。两个人分别修改同一份代码后，文件应该怎样合并？只恢复旧代码却留下新配置，是否仍是一个能运行的项目？这些问题表明，需要管理的是完整项目状态及其变化关系。

Git 将这些状态组织成可回查的历史。它并不要求先使用 GitHub：本地仓库负责记录版本，GitHub 等平台负责托管以及围绕 issue、审查和发布进行协作。

本章小结：版本管理让项目保留可理解的过去，为恢复、比较和协作提供基础。



项目把意图、规格与实现组织为可交付产物

画面证据：原视频 10:00 – 10:15，回看项目目录示意。课件把 intent、spec、impl 放在同一个项目目录中，通过构建得到应用或可执行文件。

从 U 盘与压缩包，推导出版本管理

课堂先讲代码保存在 U 盘、用邮件发压缩包的经历，再从“我想回到昨天的状态”推导出工具需求。复制目录能暂时满足回溯，但很难表达谁从哪个版本出发、哪些文件应该成组变化、两人的工作怎样汇合。

从需求出发，可以先想出一个能列出版本、切换状态的简单系统，再逐渐增加分支和合并。课堂讲述 RCS、CVS、SVN、BitKeeper 与 Git 的背景，是为了说明工具由真实协作问题推动。学习时应区分“用于理解的简化模型”与某个工具全部版本的实际存储细节，不能只凭一句“快照”或“差分”推断其所有实现。

2. 快照如何构成历史

Git 在逻辑上记录项目快照。快照描述被跟踪文件在某个时刻的内容与目录位置；提交进一步关联说明、作者和父提交。沿父提交回溯，就能看到项目如何演变。

完整快照不意味着每次物理复制所有文件。相同内容可以复用，目录对象引用已有内容，因此少量修改不需要重复保存所有未变的部分。理解这个模型，可以避免把 Git 仅看成一串压缩包。

概念	承担的作用
blob	保存文件内容的字节序列
tree	组织目录条目、名称及其引用的对象
commit	关联根目录树、父提交和提交信息

概念	承担的作用
分支	指向某个提交的可移动引用
HEAD	通常标识当前分支，也可以直接指向提交

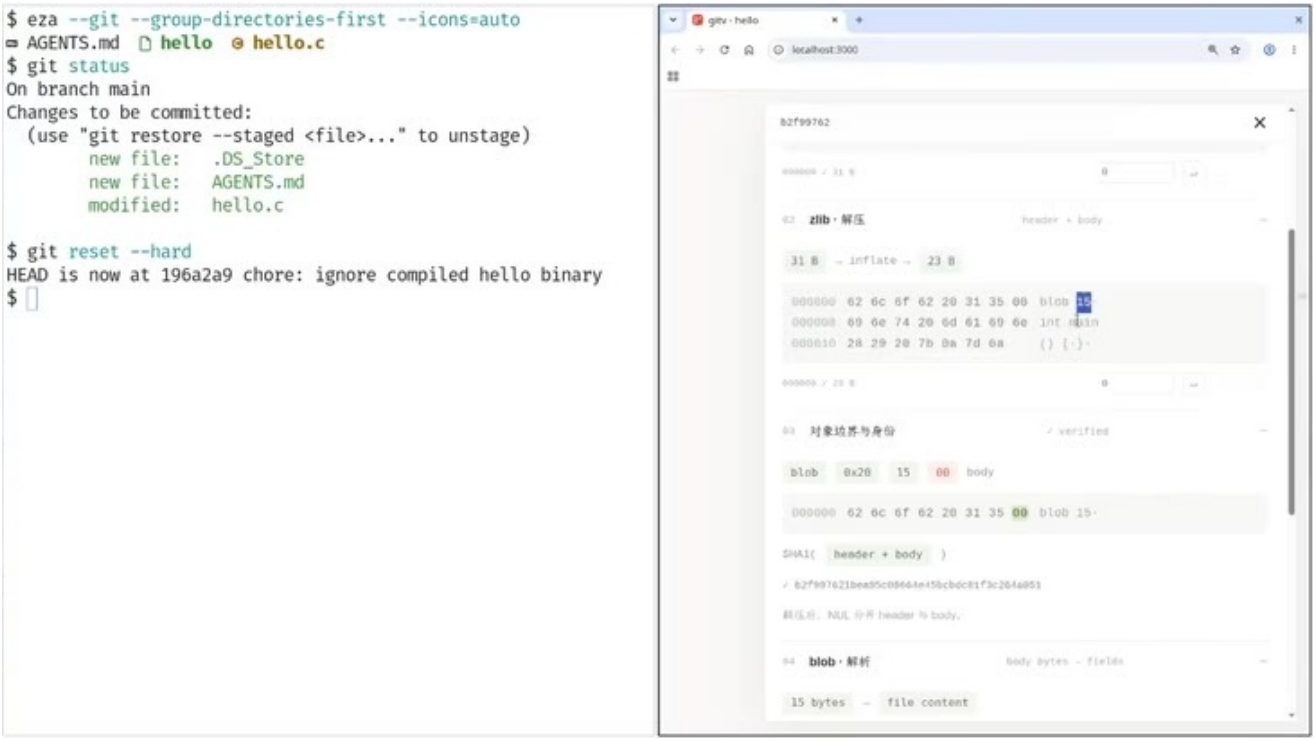
课堂使用可视化工具观察真实仓库对象。这种学习方法的价值在于把“对象”“引用”等抽象名词与磁盘中的实际数据联系起来，观察一次操作究竟改变了什么。

本章小结：Git 保存项目状态，也保存状态之间的关系；对象复用使快照可以高效存储。

持久化目录树为什么不必复制整个项目

设目录中有 A、B 和子目录 D。若只改了 B，新版本需要保存 B 的新内容，以及从 B 到根目录这条路径上发生变化的目录节点；A 和 D 可以继续指向旧对象。旧版本仍可沿旧的引用访问，新版本沿新的引用访问，这就是课堂用来说明持久化数据结构的直觉。

课堂的 gitv 工具不是仅在网页上模拟几个提交，而是读取真实仓库的文件及其原始字节，展示解压后的对象内容。这样可以检查 blob、tree 和 commit 这些名词究竟对应什么。观察对象时还要区分压缩前后大小：磁盘文件字节数、对象头和正文长度是不同量，不能把它们混为同一指标。



从真实对象文件查看 blob 字节与解析结果

画面证据：72:30 – 72:45，回看真实 Git 对象。右侧展示对象字节及解析，左侧是终端状态；图中命令是演示现场，不是要求读者照做的恢复方案。

3. 为什么要区分工作区和暂存区

工作区是正在编辑的文件，暂存区保存下一次提交准备采用的内容，提交历史保存已经记录的快照。暂存区允许开发者组织一次修改，而不是把所有正在探索的内容一次打包。

编辑补充示例：

```
git init

git add hello.c

git diff --cached

git commit -m "feat: add a minimal hello program"
```

如果执行 `git add hello.c` 后又修改了文件，普通 `git commit` 提交的是已暂存的版本。后续编辑仍留在工作区，需要再次暂存才进入这次提交。`git diff` 用于查看未暂存的修改，`git diff --cached` 用于查看已暂存的修改。

这也说明为什么不应不经检查就执行 `git add .`：它可能把编译产物、临时文件和私人配置一起纳入提交。命令合法，不代表项目选择合理。



执行 `git add` 后，`git status` 显示准备提交的 `hello.c`
画面证据：37:30 – 37:45，回看暂存区状态。界面明确写着 `No commits yet`，但 `hello.c` 已列在准备提交的变化中；这正好说明暂存与提交是两步。

本章小结：提交是经过选择和组织的变更；暂存区提供了表达这一选择的位置。

4. 分支、合并与冲突解决了什么问题

分支让不同工作可以从共同起点向前推进。一个人修改界面，另一个人修改解析器，双方先记录各自的变化，再合并结果。

Git 可以依据共同祖先与双方修改计算合并。当同一段文本出现无法自动决定的变化时，工作区通常会出现冲突标记，展示双方内容。开发者需要理解上下文，把文件改成最终需要的结果，再标记解决并完成提交。

合并成功只说明文本变化得到组合，不说明行为正确。两个分支即使修改不同文件，也可能在接口、配置或业务规则上不兼容，所以还要验证组合后的项目。

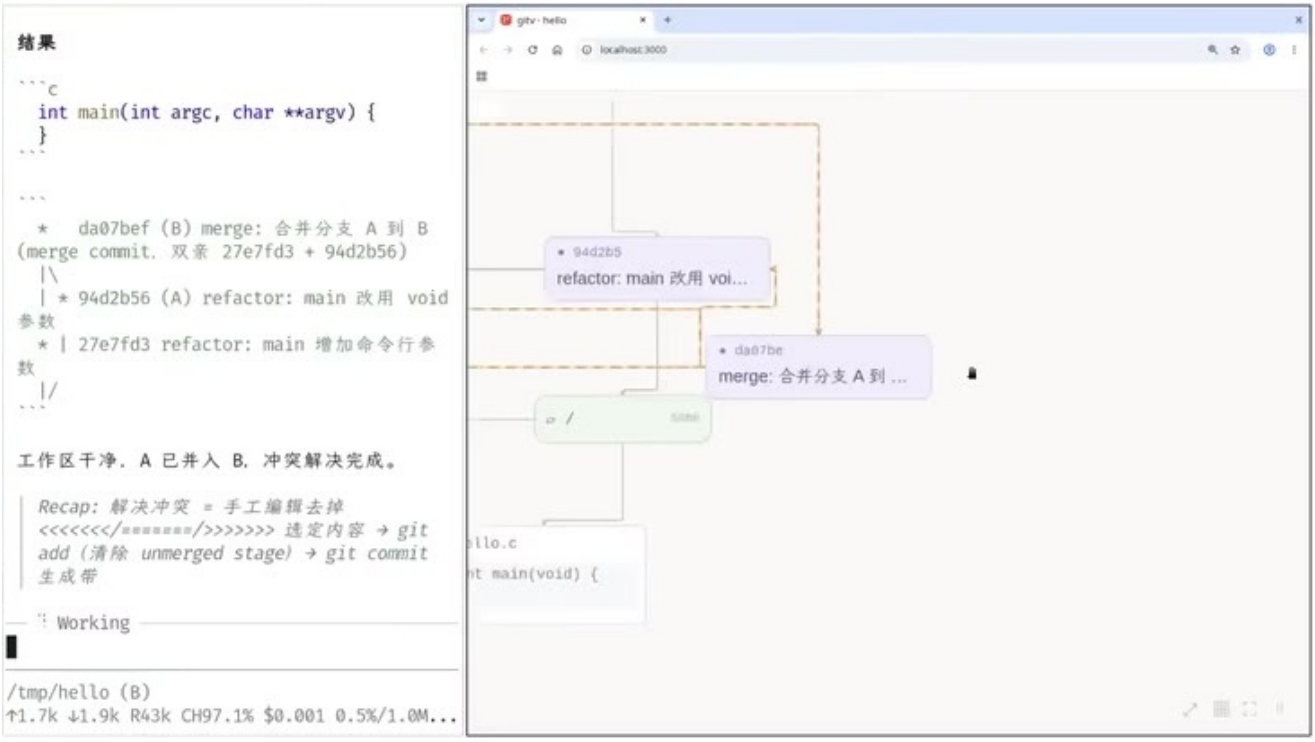
官方讲义补充说明了 rebase：它把一串改动重新应用到新的起点，通常会产生新的提交标识。是否可以整理历史取决于协作约定；别人已经依赖的提交不宜随意改写。

本章小结：分支隔离尝试，合并汇合变化；验证负责判断组合后的行为。

冲突标记之外，暂存区还记录了什么

课堂的可视化演示进一步展示了冲突时的 base、ours、theirs：共同基础和两侧内容同时参与判断。工作区中的尖括号标记让人看到冲突，但解决它并不是机械点击“保留我方”；要结合这段代码应实现的行为选择或重新编写结果。

解决冲突后，暂存区中的冲突状态消失，合并提交关联最终目录树与双方父提交。图上多了一个节点只是表象，真正重要的是它表示哪份结果，以及这份结果从哪些历史状态汇合而来。



演示结束后的合并提交与分支关系

画面证据：77:30 – 77:45，回看合并结果。左侧解释解决冲突和生成提交的流程，右侧展示仓库历史关系。

5. 怎样组织一次有意义的提交

课程最值得记住的工程原则是，把项目看作由小而完整的变更推进的一系列快照。这里的“小”首先指目的清楚，而非行数少。实现一个功能可能涉及代码、测试和文档，它们共同服务同一意图，可以组成一次完整提交。

相反，修复错误时顺手重排整个项目，会混合不同目的。将来出现故障，读者难以判断哪些变化相关，也难以单独撤销。保持提交边界清楚，可以降低审查、恢复与定位问题的成本。

提交说明应补充 diff 没有直接表达的意图。只有 update 或“做了些东西”，无法帮助后来的人理解变化。标题概括行为，正文解释动机、约束和取舍；具体格式服从项目约定。

编辑补充示例：

```
fix(parser): accept repeated spaces between arguments


Ignore extra spaces between arguments while preserving spaces inside quotes.
```

Conventional Commits 提供 type(scope): description 的一种组织方式，但格式正确不等于说明充分。代码能否构建、相关测试能否通过，同样影响这次提交能否成为可靠的历史检查点。



Thinking...

这个 commit 不太符合 best practice，主要问题在**提交信息**（其次代码本身也有问题）。

问题分析

方面	现状	Best practice
提交信息可读性	I did some cool stuffs —— 完全看不出做了什么	说明「做了什么 + 为什么」，让未来的你和协作者能看懂
格式规范	无统一前缀	建议遵循 Conventional Commits：feat: / fix: / refactor: 等
信息粒度	只是一个新文件，粒度过小	一个 commit 应是一个**逻辑完整**的变更，但新增空文件本身意义不明
代码内容	int main() {} 没有 include、没有 return	与 commit 规范无关，但属于质量问题的信号

举例，更好的写法：

```
feat: add hello program skeleton
```

Agent 对课堂提交的说明、格式和信息粒度进行检查

画面证据：40:00 – 40:15，回看提交审查。示例提交的主要问题是说明没有信息量；图中的规范建议服务于读者理解，不意味着所有仓库必须使用完全相同的格式。

空提交为什么存在，又为什么不应滥用

课堂通过没有文件变化的提交说明，提交不仅包含目录树，也有说明等元信息。有时可以借空提交触发自动流程或表达一个明确的工程事件，但如果大量空提交没有可解释目的，历史会增加噪声。发布标记也可以通过 tag 表达，选哪种形式取决于项目约定。

课堂还让 Agent 整理实验历史：它先注意到仓库没有 remote，再处理先前不合理的提交。这一细节体现了历史重写与协作的关系。别人已经基于某段历史工作时，改写它会改变共同参照，不能只因为“整理后看起来漂亮”就忽略依赖关系。

本章小结：好的提交边界让变化可以解释，好的说明让后来的人知道为什么要保留它。

6. 什么应该进入仓库

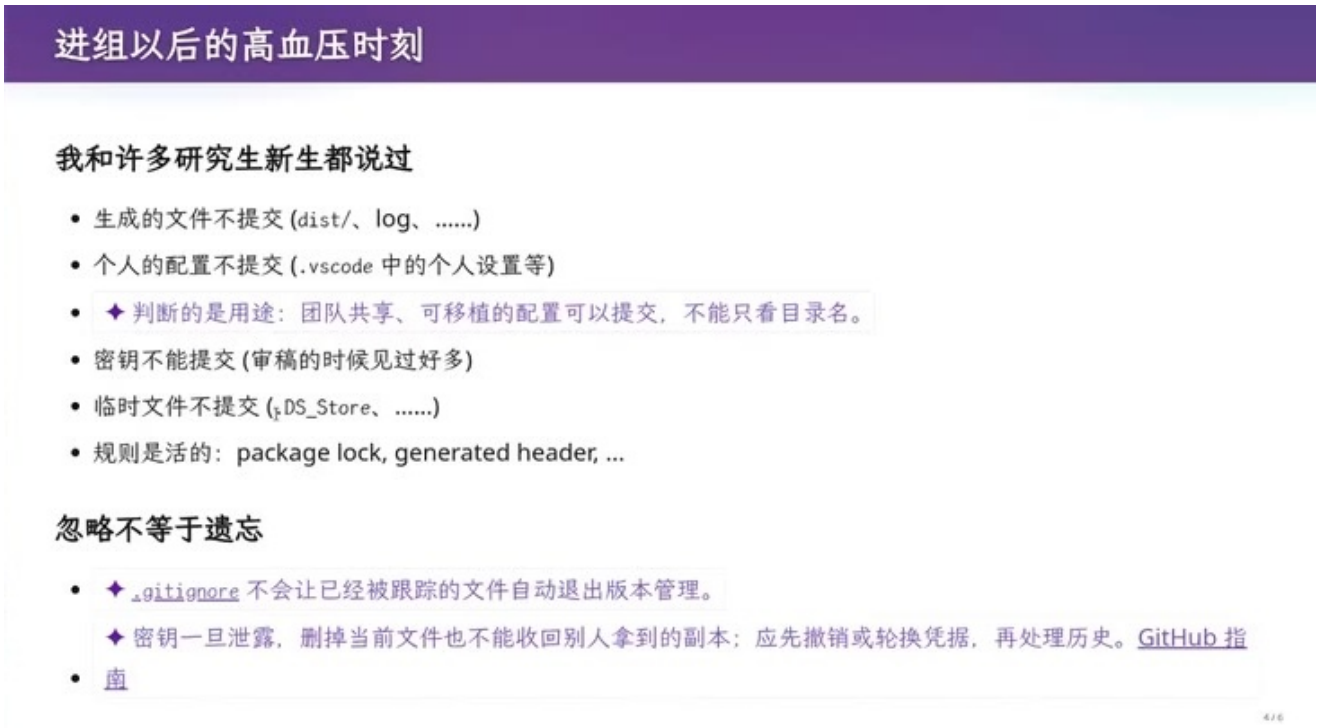
仓库保存共同维护的项目状态，不必保存工作目录里出现的一切。编译产物和日志通常能由源码及工具重新生成，不适合每次提交；机器绝对路径和个人编辑器偏好也可能让他人无法复现。

文件类型	判断依据
源码、测试、构建说明	通常是共同维护的项目内容
编译产物、运行日志	通常忽略，防止膨胀及无意义冲突
私人路径和临时文件	通常排除，避免依赖个人机器
凭据与 API key	不进入版本历史
依赖锁文件或生成头文件	依据复现需求、工具链和团队约定决定

“生成文件一律不提交”过于简单。锁文件可以固定依赖版本，某些生成头文件可以减少使用者安装额外工具链的负担。关键问题是为什么共享它，以及怎样保持一致。

官方讲义提醒：`.gitignore` 不会让已经跟踪的文件自动退出历史。误提交密钥后，仅删掉当前文件不能消除旧版本中的泄露，应按凭据类型处理失效、轮换及历史清理。

本章小结：根据文件在项目中的角色决定是否提交，不按文件名机械判断。

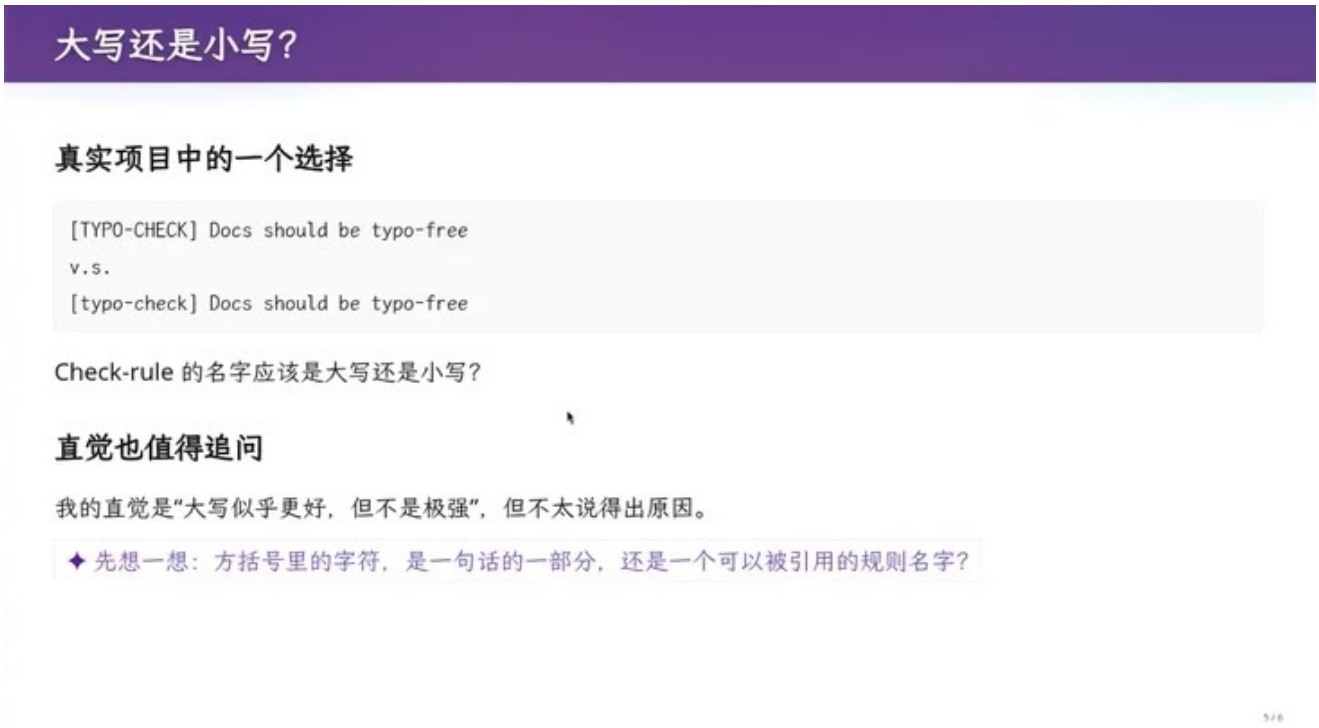


课件列举生成文件、个人配置、密钥及规则的例外
画面证据：57:30 – 57:45，回看文件选择规则。课件同时强调规则依用途判断，以及忽略文件不等于抹除已有历史。

从规则命名理解工程上的判断

课堂比较 `[TYPO-CHECK] Docs should be typo-free` 与小写标识的写法。方括号中的内容承担固定规则名的角色，后半句承担自然语言解释的角色；两者视觉上可区分，规则也更便于被引用。这个分析解释了为何有人偏好大写，并不推出“大写永远更好”。

如果工具要求小写，或项目一直使用小写，一致性可能比局部醒目更重要。细节学习的目的，是把模糊直觉还原成角色、约束和取舍，而不是积累没有前提的口号。



TYPO-CHECK 与 typo-check 的规则命名对比
画面证据：60:00 – 60:15，回看规则名讨论。

7. 历史如何帮助验证与恢复

及时保存合理的检查点，可以让失败的探索有退路。`git bisect` 利用好版本与坏版本间的检查结果缩小范围，帮助找到引入问题的提交；`git blame` 帮助追溯代码行的来源；`git revert` 则以新提交表达对既有变化的撤销。

这些工具的效果取决于历史质量。如果一个提交混合了许多独立目的，定位到它后仍需拆解；如果大量中间版本无法构建，也会增加二分检查的困难。

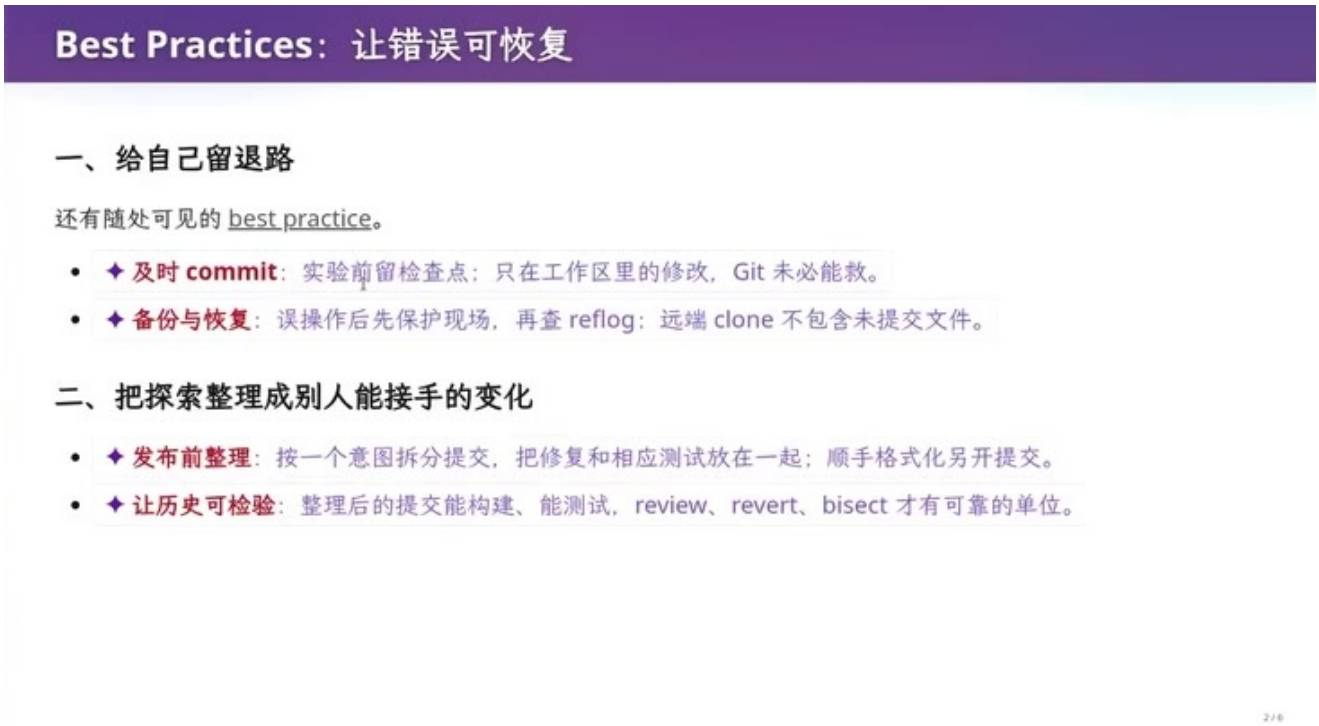
GitHub 上的 issue、PR 和检查结果属于平台协作信息，并不自动包含在 Git 的文件历史中。课堂提到的 `gh`、`glab` 可以让 Agent 通过命令行连接这些协作步骤。使用平台工具仍要区分本地验证、远端推送、提出合并申请和正式合入。

本章小结：历史不仅用来存档，也可以成为定位、审查和恢复的工具。

经常保存，与发布前整理并不矛盾

探索时先保留检查点，可以降低误操作后无处可退的风险；准备交给他人时，再在允许的范围内整理成目的清楚的变化。只存在于工作区的修改没有自动进入提交历史，远端 clone 也不会包含这些未提交内容。

课堂围绕小步提交解释了 bisect、review、revert 的价值：每一步如果可构建、可检查，后续操作就有可靠的判断单位。这是“小提交”的工程意义，而不是把一项完整修改任意拆成许多无法运行的碎片。



检查点、备份与按意图整理历史
画面证据：90:00 – 90:15，回看可恢复的工程实践。

hooks、CI 与平台工具分工

Git hooks 可以在提交等事件附近执行检查，CI 可以在集成流程中检查构建与测试。课堂还展示了 Agent 完成任务时发提示音的 hook；它是 Agent 工具的事件机制，与 Git 的 pre-commit hook 不能混成同一个东西。

规则和自动化只能检查已经表达的要求。格式检查通过，不能证明接口设计合理；分支各自测试通过，也不等于组合后的项目通过。gh、glab 把平台中的 issue、评论与 PR 操作连接到工作流程，Git 则继续负责文件版本。

8. AI 能写代码以后，人还要做什么

课程建议在架构阶段先集中讨论：明确职责、接口和约束，减少相互影响的共享状态，再把边界写成文档、规则与示例。接口稳定后，更多实现细节可以交给 Agent 独立处理。

课堂的编程可视化项目说明了这种做法。通过清晰接口和较少的核心共享状态，组件可以成为相对独立的黑盒，Agent 只需要理解接口，而不必读取所有实现。

实现阶段可以安排多个 Agent 处理相互独立的任务，但并发数量并非质量保证。任务依赖、文件覆盖、接口兼容和集成责任仍需要管理。讲师展示的是个人工作方式，不是所有项目都应遵守的并发上限。

有界面的项目应尽早运行并观察效果。只读“已完成”不能发现按钮位置、交互顺序或文字密度的问题；持续反馈可以尽早纠正设计偏差。

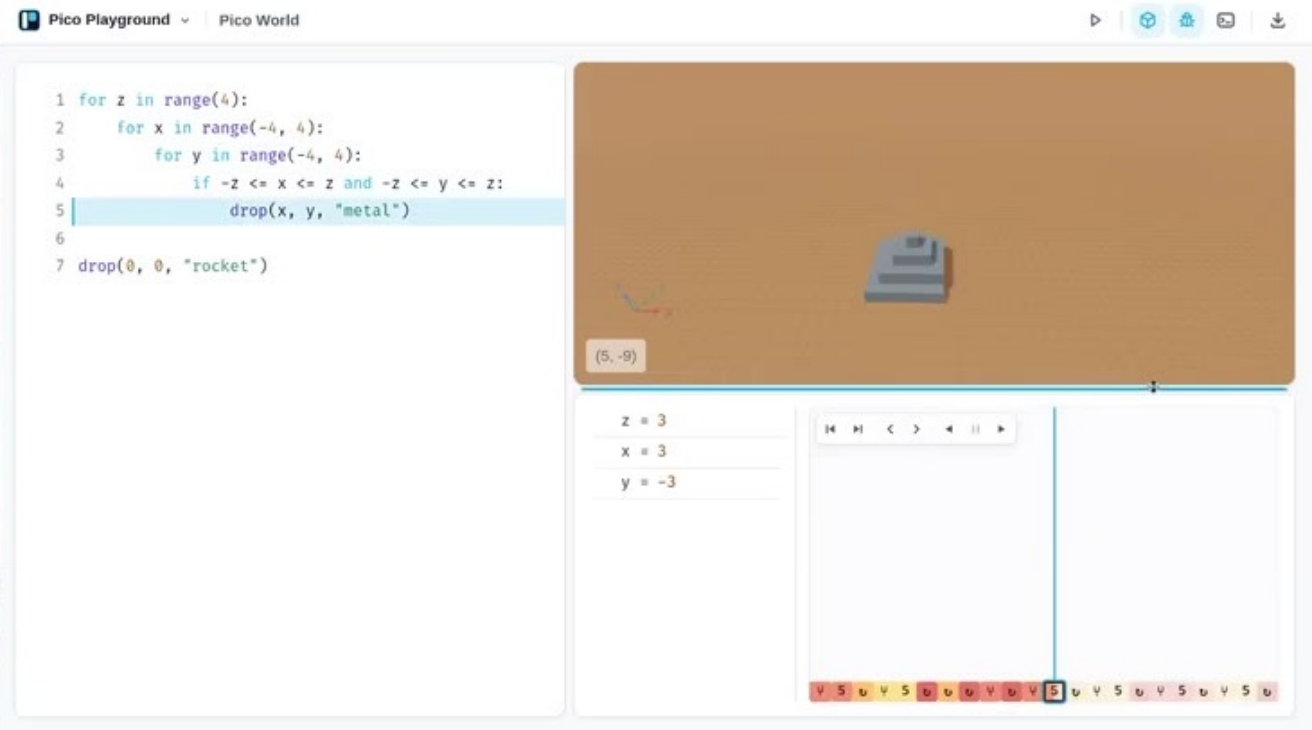
课程结尾提出进一步的问题：当生成速度远超人的理解和验收速度，版本控制与协作方式会遇到什么瓶颈？这节课提出问题，为后续讨论铺垫，不把设想写成已实现的自动化系统。

本章小结：人的贡献更多体现在定义目标、形成设计边界和判断结果，版本管理让探索过程可控。

案例：为什么儿童编程环境需要先做好架构

讲师展示了面向小学三、四年级学生的编程环境。目标是让孩子尽快看到 Python 程序的效果，而非先学习复杂 IDE 的所有配置。演示把代码、运行轨迹和可视化画面联系起来，拖动进度可以观察不同执行状态。

这样的界面看起来只是几块面板，但背后需要明确接口：程序怎样产生轨迹，轨迹怎样驱动图像，光标和播放位置怎样协调。讲师强调把尽可能多的组件设计成纯函数，把少量跨组件状态集中管理；于是实现组件的 Agent 可以只依赖接口，减少反复阅读其他实现的成本。



Pico Playground 中的程序、运行轨迹与三维结果

画面证据：82:30 – 82:45，回看编程环境。本文只说明课堂演示的架构与交互，不声称已独立复现此工具。

多个 Agent 的前提仍是边界清楚

进入实现阶段之前，先通过架构讨论形成共同前提，再用 AGENTS.md 记录规则。实现过程中及时观察真实效果、指出不足，让偏差尽快暴露。并行处理边界清楚的任务可以推进工作；不断增加终端数量，却不明确谁负责接口和集成，会把混乱转移到合并阶段。



项目由原子变更推进，以及 AI-native 项目也应避免混沌
画面证据：95:00 - 95:15，回看本讲工程总结。

结尾的思想实验：当生成速度增长一千倍

最后的课件把“快一千倍”作为放大矛盾的思想实验：如果代码生成极快，而人理解和验收的速度没有跟上，积压的会是等待判断的变更。系统需要回答哪些改动能组合、哪些有依赖、哪些组合后要重新验证。

这里的数字不是一次性能测试结果，也不表示课堂已经实现了一套新版本控制系统。它把问题引向变更组合、并发控制与准入判断，说明今天学习快照、接口和验证概念，仍能帮助理解未来的工具。



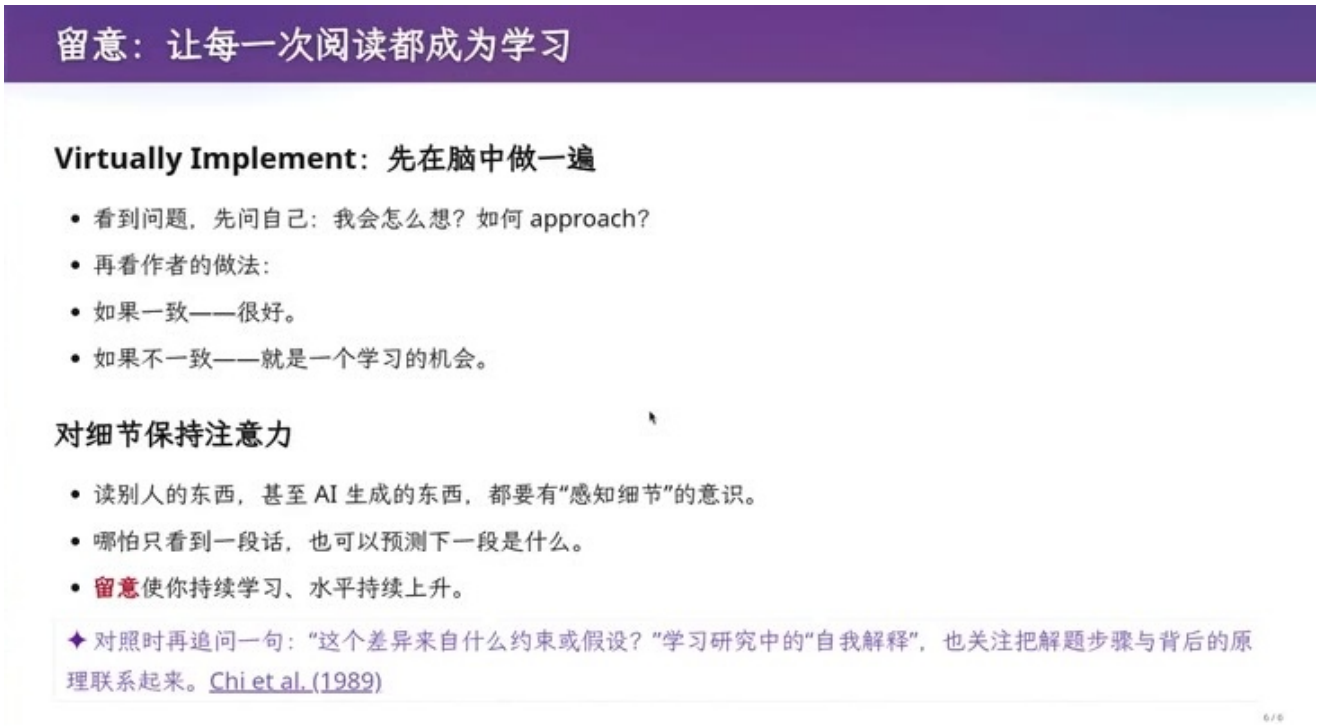
关于 Agent 产能与版本控制的结尾思想实验
画面证据：100:00 – 100:13，回看结尾课件。该画面确实在原视频末尾，本文不据此推断已经发生了后续讨论。

9. 怎样借助 AI 学习工程知识

课程反复强调注意力：看到成熟项目时，留意目录、文档和操作反馈。先想自己会如何解决，再比较现成方案；如果对方更好，追问它解决了哪个自己忽略的约束。

这种方法被称为 virtually implement。它适用于读代码、论文、README，也适用于检查 AI 生成的解释。重点是形成预测并用证据修正理解。

学习 Git 时，可以要求 Agent 在操作前解释将影响哪些文件、暂存内容与历史引用，在操作后展示状态变化。这样每次真实任务都能补充一个概念，而不只是记住一条命令。



Virtually Implement 的课堂定义与注意细节的方法
画面证据：22:30 – 22:45，回看学习方法。课堂把它延伸到阅读项目、文档乃至 AI 生成的解释。

比较成熟项目的 README 也是这种方法的例子。长文档不必然信息充分，短文档也不必然清楚。应该观察读者进入项目时有哪些问题，入口是否能让人迅速找到下一步需要的内容。课堂用路口标识作类比，强调把所需信息放在产生不确定性的地方。

小结与练习

项目的演变应能被分解为可理解的变更。快照提供可恢复状态，分支和合并组织协作，清楚的提交边界与验证让历史真正有用。

练习：在一次性练习仓库中，修改文件后先暂存，再修改一次，比较两个 diff；创建两条修改同一行的分支，观察冲突；检查自己的项目中是否存在不应跟踪的产物。让 Agent 先解释预期结果，再执行并核对。

来源与许可

- 原课程：蒋炎岩，南京大学《生成式软件工程》2026 年第 3 讲，视频 https://www.bilibili.com/video/BV1kybV6DE47/。
- 官方讲义《版本管理》。
- 公开字幕整理稿，来源仓库尚未将本讲标记为完成人工校对。
- 原课程与字幕整理来源采用 CC BY-NC 4.0。本文对相关内容作删减、重排和解释，保留署名并按相同条件提供对应改编内容，不代表讲师或学校认可。

核验材料见仓库 evidence/，包含媒体哈希、字幕、配图来源和覆盖表。按 5 分钟窗口抽查并核对 12 幅原图，未作逐字、逐帧全检或课堂工具复现。